

ALGORITMOS DE BÚSQUEDA EN ARREGLOS UNIDIMENSIONALES Y BIDIMENSIONALES

Trabajo de recolección de información e implementación en PHP

Estudiante: Samantha Moreno

Carrera: Ingeniería de Sistemas

Semestre: Tercer semestre

Lenguaje utilizado: PHP

1. Introducción

En programación es muy común necesitar encontrar un dato dentro de un conjunto de elementos. Por ejemplo, buscar el número de un estudiante, un producto o un valor específico. Para realizar esta tarea existen algoritmos de búsqueda, que son procedimientos ordenados que indican cómo localizar un elemento dentro de una estructura de datos.

En este trabajo se estudian algoritmos que pueden aplicarse a arreglos unidimensionales y bidimensionales. Se explican principalmente la búsqueda secuencial, la búsqueda binaria y la búsqueda por salto para arreglos unidimensionales, además de la búsqueda secuencial aplicada a matrices. Los ejemplos fueron implementados en PHP para observar su funcionamiento práctico.

2. Objetivos

2.1 Objetivo general

Investigar y comprender algoritmos de búsqueda que puedan aplicarse sobre arreglos unidimensionales y bidimensionales, implementándolos mediante ejemplos en PHP.

2.2 Objetivos específicos

- Explicar qué es un algoritmo de búsqueda y cuál es su utilidad.
- Diferenciar arreglos unidimensionales y bidimensionales.
- Comprender el funcionamiento de la búsqueda secuencial, binaria y por salto.
- Aplicar una búsqueda mediante filas y columnas en una matriz.
- Implementar ejemplos funcionales en PHP.
- Comparar las ventajas, desventajas y complejidad de los algoritmos estudiados.

3. Fundamento teórico

3.1 ¿Qué es un algoritmo de búsqueda?

Un algoritmo de búsqueda es un conjunto de pasos utilizados para encontrar un elemento determinado dentro de un conjunto de datos. El objetivo es saber si el elemento existe y, cuando es posible, conocer su posición.

3.2 ¿Qué es un arreglo?

Un arreglo es una estructura que permite almacenar varios datos relacionados y acceder a ellos mediante posiciones o índices. En PHP, los arreglos se representan con la estructura array.

3.3 Arreglo unidimensional

Un arreglo unidimensional puede imaginarse como una sola fila de casilleros. Cada elemento ocupa una posición. En PHP los índices normalmente comienzan en cero.

Ejemplo: [10, 20, 30, 40, 50]. En este caso, el valor 30 se encuentra en el índice 2.

3.4 Arreglo bidimensional

Un arreglo bidimensional está formado por filas y columnas. Puede representarse como una tabla o matriz. Para localizar un elemento se necesita indicar la fila y la columna.

Ejemplo: una matriz de 3 filas por 3 columnas puede contener los valores 10, 20, 30, 40, 50, 60, 70, 80 y 90.

4. Algoritmos de búsqueda

4.1 Búsqueda secuencial o lineal

La búsqueda secuencial revisa los elementos uno por uno, comenzando desde el primero. Si encuentra el valor buscado, devuelve su posición. Si llega al final sin encontrarlo, indica que el elemento no existe.

Ventaja: es sencilla y funciona aunque los datos no estén ordenados. **Desventaja:** puede revisar muchos elementos. Su complejidad en el peor caso es $O(n)$.

4.2 Búsqueda binaria

La búsqueda binaria aprovecha que los datos están ordenados. Compara el elemento buscado con el valor central. Si el buscado es mayor, continúa en la mitad derecha; si es menor, continúa en la mitad izquierda. Repite el proceso hasta encontrarlo o quedarse sin elementos.

Ventaja: es mucho más rápida para conjuntos grandes. **Desventaja:** requiere que el arreglo esté ordenado. Su complejidad es $O(\log n)$.

4.3 Búsqueda por salto

La búsqueda por salto también necesita datos ordenados. En lugar de revisar cada elemento, avanza en bloques. Cuando encuentra el bloque donde podría estar el objetivo, realiza una búsqueda secuencial dentro de ese bloque.

Su complejidad típica es $O(\sqrt{n})$. Puede ser útil cuando se quiere reducir la cantidad de comparaciones respecto de una búsqueda secuencial.

4.4 Búsqueda en arreglos bidimensionales

Una forma sencilla de buscar en una matriz es recorrer cada fila y, dentro de ella, cada columna. Para esto se utilizan dos ciclos: uno controla la fila y otro controla la columna. La búsqueda termina cuando se encuentra el elemento.

Si la matriz tiene F filas y C columnas, una búsqueda secuencial puede realizar hasta $F \times C$ comparaciones, por lo que su complejidad es $O(F \cdot C)$.

5. Ejemplos

5.1 Búsqueda secuencial

Arreglo: [10, 25, 8, 40, 15, 30]. Objetivo: 40. El algoritmo compara 10, luego 25, luego 8 y finalmente 40. El elemento se encuentra en el índice 3.

5.2 Búsqueda binaria

Arreglo ordenado: [10, 20, 30, 40, 50, 60, 70, 80]. Objetivo: 70. El algoritmo revisa posiciones centrales y descarta las partes donde el valor no puede estar hasta localizar 70.

5.3 Búsqueda por salto

Arreglo ordenado: [10, 20, 30, 40, 50, 60, 70, 80, 90]. Objetivo: 70. El algoritmo avanza por bloques y luego busca dentro del bloque correspondiente.

5.4 Búsqueda en matriz

Matriz: [[10,20,30],[40,50,60],[70,80,90]]. Objetivo: 50. Se recorre fila por fila. El elemento se encuentra en fila 1 y columna 1 cuando los índices comienzan desde cero.

6. Implementación en PHP

El repositorio contiene cuatro programas independientes. Cada programa define una función para realizar la búsqueda, crea los datos de prueba, ejecuta la función y muestra el resultado.

La búsqueda binaria y la búsqueda por salto utilizan arreglos ordenados. La búsqueda secuencial y la búsqueda de la matriz no necesitan que los datos estén ordenados.

6.1 Organización del repositorio

```
algoritmos-busqueda/  
■■■ README.md  
■■■ unidimensional/  
■ ■■■ busqueda_secuencial.php  
■ ■■■ busqueda_binaria.php  
■ ■■■ busqueda_salto.php  
■■■ bidimensional/  
■■■ busqueda_matriz.php
```

Para ejecutar los ejemplos se puede utilizar PHP desde la terminal con comandos como: `php unidimensional/busqueda_secuencial.php`. También se puede utilizar el servidor integrado de PHP.

7. Comparación

Algoritmo	Ordenado	Idea principal	Complejidad
Secuencial	No	Revisar uno por uno	$O(n)$
Binaria	Sí	Dividir por la mitad	$O(\log n)$
Por salto	Sí	Avanzar por bloques	$O(\sqrt{n})$
Matriz secuencial	No	Recorrer filas y columnas	$O(F \cdot C)$

8. Ventajas y desventajas

Búsqueda secuencial: fácil de entender e implementar; funciona con datos desordenados. Su principal desventaja es que puede ser lenta cuando hay muchos elementos.

Búsqueda binaria: muy eficiente; sin embargo, necesita que el arreglo esté ordenado y correctamente organizado.

Búsqueda por salto: reduce comparaciones frente a una búsqueda lineal en determinados casos; también necesita datos ordenados.

Búsqueda en matriz: es sencilla y adecuada para matrices pequeñas; en matrices grandes puede requerir muchas comparaciones.

9. Conclusiones

1. Los algoritmos de búsqueda permiten localizar información dentro de estructuras de datos de manera organizada.
2. La búsqueda secuencial es la más sencilla y puede utilizarse cuando los datos no están ordenados.
3. La búsqueda binaria es más eficiente que la secuencial cuando el arreglo está ordenado, porque elimina aproximadamente la mitad de las posibilidades en cada paso.
4. La búsqueda por salto representa una alternativa que avanza por bloques y posteriormente realiza una búsqueda dentro del bloque adecuado.
5. En arreglos bidimensionales, una estrategia básica y fácil de comprender consiste en recorrer las filas y columnas mediante ciclos.
6. La implementación en PHP permitió comprobar de forma práctica los conceptos estudiados y observar cómo cada algoritmo encuentra un elemento y devuelve su posición.
7. Los objetivos planteados fueron cumplidos, ya que se investigaron, explicaron, compararon e implementaron algoritmos de búsqueda para arreglos unidimensionales y bidimensionales.

10. Bibliografía

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

PHP Documentation Group. (s. f.). Arrays. PHP Manual.
<https://www.php.net/manual/en/language.types.array.php>

PHP Documentation Group. (s. f.). Control structures. PHP Manual.
<https://www.php.net/manual/en/language.control-structures.php>